

B11902009

貝同光

Q1

$T(n) = T(n/2) + O(1)$. Unrolling: after k halvings the array has $n/2^k$ elements; we stop when $n/2^k = 1$, i.e. $k = \log_2 n$. Each level does $O(1)$ work, so $T(n) = O(\log n)$. Worst case: the element is absent, and we probe until the range is empty — still $\log_2 n + 1$ probes.

B11902009

貝同光

Q2

Exchange argument: let $G = g_1..g_k$ be greedy's picks (by finish time) and $OPT = o_1..o_m$ an optimal solution, both sorted. By induction $f(g_i) \leq f(o_i)$: greedy picks the earliest-finishing compatible interval, so we can swap o_i for g_i without breaking compatibility. Hence $k \geq m$, so greedy is optimal.

B11902009

貝同光

Q3

Let $L[i]$ = length of the LIS ending exactly at $a[i]$.

$L[i] = 1 + \max\{ L[j] : j < i, a[j] < a[i] \}$ (or 1 if none). Answer $\max L[i]$.

Two nested loops give $O(n^2)$. Refinement: keep $tails[k]$ = smallest tail of an increasing subsequence of length $k+1$; binary-search each element: $O(n \log n)$.

B11902009

貝同光

Q4

False: BFS can take a longer path if it enqueues vertices in a bad order,
e.g. a diamond graph where the right branch is explored first.